Tracce progetto di gruppo

Fondamenti di Robotica

Docenti: Loredana Zollo
Clemente Lauretti
Tutor: Rita Molle

email: l.zollo@unicampus.it
c.lauretti@unicampus.it
rita.molle@unicampus.it

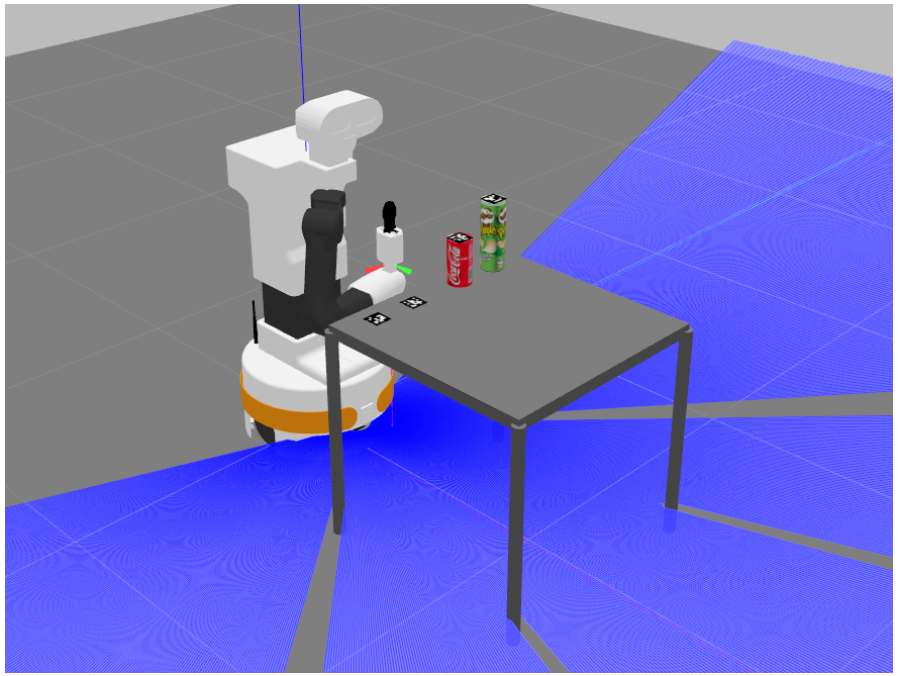
CAMPUS BIO-MEDICO UNIVERSITY OF ROME
Via Álvaro del Portillo, 21 - 00128 Rome - Italy
www.unicampus.it

CREO LAB

UNIVERSITA' CAMPUS BIO-MEDICO DI ROMA

Overview del progetto di gruppo

Obiettivo: Sviluppare un framework di simulazione che consenta al robot TIAGo di eseguire attività di pick and place, sfruttando algoritmi avanzati basati sull'intelligenza artificiale per la modellazione, la pianificazione del movimento, il controllo e la percezione del robot



Tools & conoscenze richieste: ROS 2, simulatori Rviz e Gazebo, basi di robotica, intelligenza artificiale e ArUco marker

Sfide principali:

Task 1: Modellazione di robot in Gazebo
Localizzazione di oggetti
Trasformazioni tra frame

Task 2: Pianificazione del movimento
Inversione cinematica
Controllo del movimento

Modalità di verifica

- Consegna dei pkg creati
- Presentazione di un Power Point a fine corso e domande individuali (prima del primo appello d'esame)

Consegna Task 1	Consegna Task 2
06/05/2026	28/05/2026

Sostituire il file launch

Aprire la cartella: ~/tiago_public_ws/src/tiago_simulation/tiago_gazebo/launch

Sostituire il file **tiago_gazebo.launch.py** con il file launch (stesso nome) dell'elearning

Modifiche → sostituzione del world per inserire "pick_place_close_demo"
→ aggiunta dei comandi per aprire rviz

Scaricare i modelli

TO DO

Aprire la cartella: ~/tiago_public_ws/src/pal_gazebo_worlds/models

Scaricare i modelli della cartella **Model** dell'elearning e sostituirli nella cartella **models** del ws

Modifiche → creazione ArUco Marker sul tavolo
→ creazione oggetti (pringles e Coca Cola) con ArUco Marker attaccati

Scaricare il mondo

TO DO

Aprire la cartella: ~/tiago_public_ws/src/pal_gazebo_worlds/world

Scaricare il mondo della cartella **world** dell'elearning e sostituirlo nella cartella **world** del ws

Modifiche → creazione ambiente per pick and place → tavolo con sopra ArUco e oggetti con ArUco

Build dell'ambiente

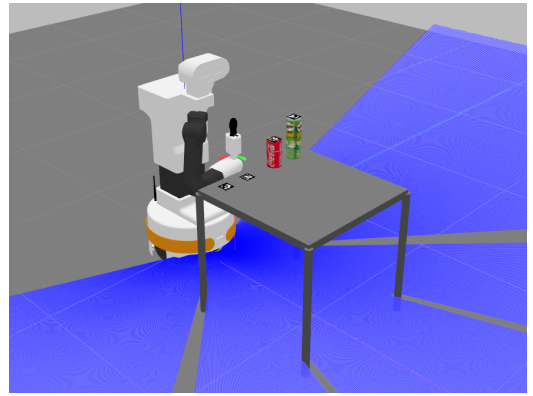
```
colcon build --packages-select pal_gazebo_worlds tiago_simulation
```

Source del workspace

```
source ~/tiago_public_ws/install/setup.bash
```

Simulazione → TIAGo nell'ambiente per pick and place:

```
ros2 launch tiago_gazebo tiago_gazebo.launch.py is_public_sim:=True
```



Task 1 – consegna 06/05/2026

Obiettivi specifici:

Programmare il robot TIAGo per i) **eseguire una scansione dell'ambiente** muovendo gradualmente la testa del robot, ii) **rilevare i marker ArUco** tramite la camera RGB-D e iii) **trasformare le coordinate dei marker** dal sistema di riferimento della camera al sistema di riferimento della base del robot.

Step da seguire:

- Avviare la simulazione del robot in Gazebo
- Movimentare la testa del TIAGo al fine di scansionare l'ambiente
- Rilevare la presenza di ArUco marker nella scena
- Stimare la posa dei marker rispetto alla terna camera
- Stimare la posa della camera rispetto alla terna base del robot
- Effettuare una trasformazione di coordinate per esprimere la posa dei marker in terna base
- Stampare la posa dei marker rispetto alla base del robot
- Visualizzare su Rviz e su gazebo la terna dei marker

Strumenti e librerie da utilizzare:

- Nodi, Topics, Services, Actions, Launch files, TF2ROS, OpenCV

Task 1 – TO DO entro il 06/05/2026

TO DO per la consegna:

- Movimentare la testa del TIAGo al fine di scansionare l'ambiente, scegliendo come fare la scansione, con quale configurazione dei giunti, etc
- Rilevare TUTTI gli ArUco marker in terna camera e in terna base
- Scrivere un launch file per avviare i nodi
- Scrivere un README che faccia comprendere quali comandi runnare da terminale

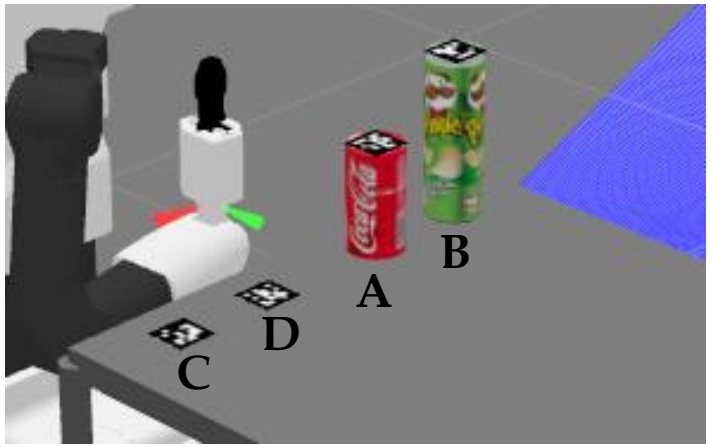
- Entro il 29/04, caricare il package (SOLO IL PACKAGE, NON IL WORKSPACE) su elearning (le istruzioni specifiche verranno fornite in seguito per email)

Suggerimenti utili per Task 1

1. **Avviare la simulazione del robot in ambiente Gazebo**
   - Seguire le indicazioni fornite alla fine della presentazione per l'installazione dei pkg del Tiago e la configurazione dell'ambiente
   - Eseguire il commando: ros2 launch tiago_gazebo tiago_gazebo.launch.py is_public_sim:=True ()
2. **Movimentare la testa del TIAGo al fine di scansionare l'ambiente**
   - Action su cui pubblicare: /head_controller/follow_joint_trajectory
   - Tipo messaggio: FollowJointTrajectory
   - 2 giunti: head_1_joint (destra/sinistra) e head_2_joint (su/giù)
3. **Stimare la posa degli ArUco presenti nella scena**
   - Leggere dal topic /head_front_camera/rgb/image_raw (tipo messaggio Image) le immagini visualizzate dal robot
   - Sottoscrivere ai parametri della camera /head_front_camera/rgb/camera_info (tipo messaggio CameraInfo)
   - Pubblicare la posa e l'ID dell'ArUco marker (cv2.aruco.DICT_6x6_250, marker_size = 0.06)
4. **Effettuare una trasformazione di coordinate per esprimere la posa dei marker in terna base**
   - Trasformare la posa dal frame 'head_front_camera_color_optical_frame' a 'base_footprint' utilizzando tf
   - Pubblicare su uno o più topic la posa degli ArUco rilevati

Task 2 – consegna 28/05/2026

Obiettivi specifici:
Programmare il robot TIAGo per i) raggiungere con l'organo terminale due oggetti nello spazio di lavoro (posizione A e B in figura), ii) afferrarli con la pinza, e iii) spostarli rispettivamente in posizione C e D. Le posizioni iniziali e finali degli oggetti sono definite dagli Aruco marker.



Moduli da sviluppare

- Pianificatore del movimento nello spazio operativo
- Inversione cinematica con gestione degli ostacoli
- Modulo per invio comandi al controllo nello spazio dei giunti (tronco + braccio + gripper)

Step da seguire:

- Avviare la simulazione del robot in Gazebo
- Caricare il file urdf del TIAGo (contenente solo alcuni giunti e alcuni link) tramite robotics toolbox
- Portare il robot in una configurazione che consenta di eseguire il task
- Visualizzare la posa degli aruco e dell'end-effector su RViz per pianificare la pose da raggiungere
- Scegliere una sequenza di movimentazione degli oggetti che agevoli l'evitamento degli ostacoli (gestire la sequenza delle operazioni)
- Pianificazione e inversione cinematica utilizzando le funzioni del robotics toolbox e invio della configurazione dei giunti desiderata al controller

Strumenti e librerie da utilizzare:

- Nodi, Topics, Services, Actions, Launch files, RoboticsToolbox, Rviz, Gazebo

Task 2 – TO DO entro il 28/05/2026

TO DO per la consegna:

- Portare il braccio in una posizione comoda per eseguire ogni pick
- Aprire il gripper prima di eseguire il pick
- Fare il pick and place di ogni oggetto in base alla configurazione del gripper che si ritiene più opportuna (oggetto preso dall'alto, lateralmente, frontalmente; oggetto posato dall'alto, lateralmente, frontalmente)
- Gestire la sequenza tramite degli stati definiti all'interno del nodo
- Scrivere un launch file per avviare sia i nodi del task 1 che i nodi del task 2, in sequenza
- Scrivere un README che faccia comprendere quali comandi runnare da terminale

- Entro il 20/05, caricare il package (SOLO IL PACKAGE, NON IL WORKSPACE) su elearning (le istruzioni specifiche verranno fornite in seguito per email)

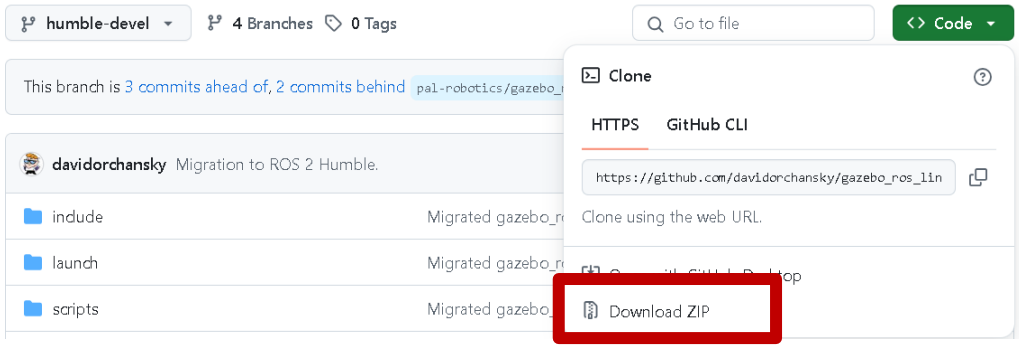
Suggerimenti utili per Task 2

1. **Caricare il file urdf del TIAGo tramite le funzioni di RoboticsToolbox**
   - Scaricare il file URDF da e-learning
   - Inserire il file appena scaricato in un pkg «my_robot_description\urdf» all'interno del workspace del progetto
2. **Portare il robot in una configurazione che consenta di eseguire il task**
   - Action su cui pubblicare: /arm_controller/follow_joint_trajectory ; /torso_controller/follow_joint_trajectory; /gripper_controller/follow_joint_trajectory
   - Tipo messaggio: JointTrajectory
   - OPPURE... Service play_motion2 "unfold_arm"
3. **Visualizzare la posa degli aruco e dell'end-effector su RViz**
   - Le pose da raggiungere non saranno esattamente quelle individuate dagli Aruco, ma avranno degli offset su posizione e orientamento (per la pianificazione degli offset aiutarsi con la visualizzazione delle terne in Rviz)
4. **Scegliere una sequenza di movimentazione degli oggetti che agevoli l'evitamento degli ostacoli**
5. **Pianificazione, inversione cinematica e invio comandi al controller**
   - Implementare un nodo che riceve la posa desiderata, pianifica il movimento nello spazio operativo, inverte la cinematica e invia la traiettoria nello spazio dei giunti al controller tramite action o publisher

   NB: nel caso in cui il tipo di task è "chiusura/apertura gripper" il movimento verrà pianificato direttamente nello spazio dei giunti

Suggerimenti utili per Task 2 – Spostare oggetto

1. **Scaricare zip gazebo_ros_link_attacher**
   - https://github.com/davidorchansky/gazebo_ros_link_attacher/tree/humble-devel
   - Cliccare su «Code» e scaricare il file zip



2. **Estrarre il file zip; creare un nuovo ws *gazebo_link_attacher_ws* e src nel ws**
3. **Entrare nella cartella src del ws e copiare la cartella appena estratta**
   - cd gazebo_link_attacher_ws
4. **Fare il build e source**
   - colcon build
   - source install/setup.bash
5. **Aprire il bashrc e copiare l'export alla fine del file CAMBIANDO il path e mettendo il proprio relativo alla cartella /install/gazebo_ros_link_attacher/lib**
   - gedit ~/.bashrc
   - source ~/gazebo_link_attacher_ws/install/setup.bash
   - export GAZEBO_PLUGIN_PATH=$GAZEBO_PLUGIN_PATH:/home/rita/gazebo_ros_link_attacher_ws/src/gazebo_link_attacher/install/gazebo_ros_link_attacher/lib

Suggerimenti utili per Task 2 – Spostare oggetto

1. **Attaccare oggetto (da terminale)**
   - ros2 service call /attach gazebo_ros_link_attacher/srv/Attach "{model_name_1: 'tiago', link_name_1: 'gripper_left_finger_link', model_name_2: 'cocacola', link_name_2: 'link'}"

2. **Staccare oggetto (da terminale)**
   - ros2 service call /detach gazebo_ros_link_attacher/srv/Attach "{model_name_1: 'tiago', link_name_1: 'gripper_left_finger_link', model_name_2: 'cocacola', link_name_2: 'link'}"

Step

- Raggiungere l'oggetto con il gripper
- Chiudere il gripper mandando il comando [0.041, 0.041] a gripper_controller/follow_joint_trajectory
- Attaccare l'oggetto con service del punto 1
- Spostare l'oggetto nella posizione di place
- Aprire il gripper mandando il comando [0.044, 0.044] a gripper_controller/follow_joint_trajectory
- Staccare l'oggetto con service del punto 2
- Tornare nella posizione iniziale

Inserire il comando attacca/stacca nel nodo .py di pick and place